

# Java代码审计基础-Servlet核心技术

---

## 目录大纲

- Servlet基本概念
    - 什么是Servlet
    - 为什么要使用Servlet
    - 如何使用Servlet
    - 实现Servlet的三种方式
  - Servlet的核心技术
    - Servlet生命周期（工作流程）
    - Servlet的HTTP对象
    - Servlet四大域对象
    - Filter过滤器技术
  - Servlet的几点疑问
    - Servlet是单例模式吗？
    - Servlet是线程安全吗？
    - Servlet中如何处理多线程安全问题？
- 

Servlet是一项古老的技术，但它仍是Java中最重要的技术之一

## 一、Servlet基本概念

### 1.1 什么是Servlet

Servlet是为了处理JavaWeb的HTTP请求、响应的程序，遵循Servlet规范，运行在服务器。

例如Tomcat是JavaWeb的服务器，本质是一个Servlet容器，负责处理客户端请求，将请求转发给Servlet，将Servlet响应转发给客户端。

Servlet是一套JavaWeb的开发规范，负责处理HTTP的请求与响应

目前实现了Servlet规范的Web服务器有Tomcat、WebLogic、JBoss、WebSphere等产品

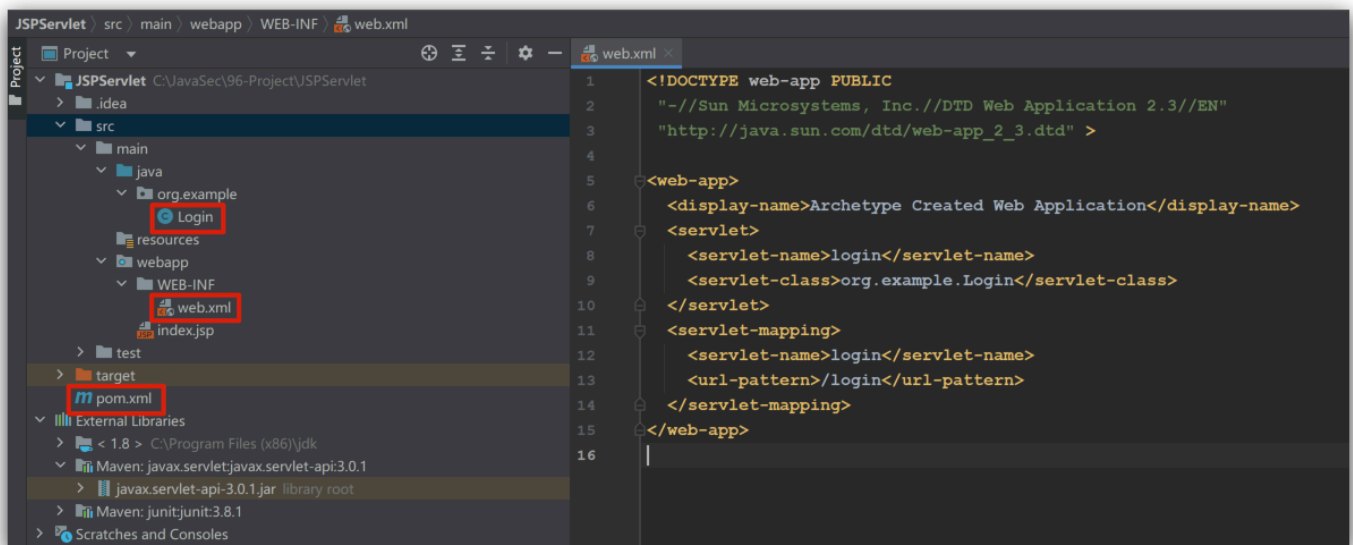
| Servlet版本   | 发布时间     | JAVAEE / JDK版本     |
|-------------|----------|--------------------|
| Servlet 4.0 | 2017年10月 | JavaEE 8           |
| Servlet 3.1 | 2013年5月  | JavaEE 7           |
| Servlet 3.0 | 2009年12月 | JavaEE6 / Java SE6 |
| .....       | .....    | .....              |
| Servlet 1.1 |          | JDK 1.1            |
| Servlet 1.0 | 1997年6月  |                    |

## 1.2 为什么要用Servlet

Java在1996年发布JDK1.0时，只实现了核心的API、JVM等核心内容，并未支持Web，在JDK1.1时才支持了Servlet标准，是Java最初实现Web服务的技术标准

## 1.3 如何使用Servlet

在Maven未出现之前，需要下载Servlet的Jar文件，引入到JavaWeb项目中使用。现在使用Maven管理依赖更加方便，一个Maven创建的JavaWeb项目的结构如图所示（项目名：JSPServlet）



web.xml是JavaWeb程序主要的配置文件

pom.xml中存放Maven的依赖管理

具体的项目构建过程可以参考【[Java代码审计环境搭建-IDEA安装配置及运行Web环境](#)】

## 1.4 实现Servlet的三种方式

- 实现javax.servlet.Servlet接口
- 继承javax.servlet.GenericServlet类
- 继承javax.servlet.http.HttpServlet类

```

1  /**
2   * 抽象接口，定义Servlet生命周期
3   */
4  public interface Servlet {
5      init();
6      service();
7      destroy();
8      // 获取Servlet的配置对象，对应的是web.xml中的servlet>标签下的<init-param>
9      getServletConfig();
10 }
11 /**
12  * 扩展Servlet，是与协议无关的Servlet
13  */
14 public abstract class GenericServlet implements Servlet, ServletConfig,
    Serializable {
15     init();
16     service();
17     destroy();
18     getServletConfig();
19     // 扩展方法
20     getInitParameter();
21     getServletConfig();
22     getInitParameterNames();
23     getInitParameter();
24 }
25 /**
26  * HTTP协议的Servlet
27  */
28 public abstract class HttpServlet extends GenericServlet {
29     init();
30     service();
31     destroy();
32     getInitParameter();
33     getServletConfig();
34     getInitParameterNames();
35     getInitParameter();
36     // 扩展方法
37     doGet();
38     doPost();
39     doHead();
40     doPut();
41     .....
42 }

```

上面的代码摘自JDK，可以看出继承关系为：**Servlet** -> **GenericServlet** -> **HttpServlet** -> 自定义Servlet

## 二、Servlet的核心技术

### 2.2 Servlet生命周期（工作流程）

1. init()负责初始化，只执行一次；在父类中只是一个空方法，需要自定义Servlet自己重写
2. service()负责处理HTTP请求
3. destroy()在Servlet销毁时执行，例如Tomcat关闭时

### 2.3 Servlet的HTTP对象

HttpServletRequest对象相关类与接口

```
1 public interface ServletRequest {}
2 public interface HttpServletRequest extends ServletRequest {}
3
4 public class ServletRequestWrapper implements ServletRequest {}
5 public class HttpServletRequestWrapper extends ServletRequestWrapper implements
  HttpServletRequest {}
```

HttpServletResponse对象相关类与接口

```
1 public interface ServletResponse {}
2 public interface HttpServletResponse extends ServletResponse {}
3
4 public class ServletRequestWrapper implements ServletRequest {}
5 public class HttpServletRequestWrapper extends ServletRequestWrapper implements
  HttpServletRequest {}
```

可以看到HttpServletRequest对象、HttpServletResponse对象都使用了 **装饰模式**，为什么？

假设现在有个场景，需要在HTTP参数到达Servlet之前进行参数的过滤，而HttpServletRequest中并没有提供对应的方法，只有一个getParameter(String name)可以获取参数，所以，可以通过自定义过滤器的方式实现。使用刚开始的Servlet项目，做如下修改：

自定义ParamFilter类

```
1 package org.example;
2
3 import javax.servlet.*;
4 import javax.servlet.http.HttpServletRequest;
5 import javax.servlet.http.HttpServletRequestWrapper;
6 import java.io.IOException;
7 import java.util.Map;
8
9 public class ParamFilter implements Filter {
10     public void init(FilterConfig filterConfig) throws ServletException {
11
12     }
```

```

13
14     public void doFilter(ServletRequest servletRequest, ServletResponse
15 servletResponse, FilterChain filterChain) throws IOException, ServletException {
16         ModifyParametersWrapper mParametersWrapper = new
17 ModifyParametersWrapper((HttpServletRequest) servletRequest);
18         filterChain.doFilter(mParametersWrapper, servletResponse);
19     }
20
21     public void destroy() {
22     }
23
24     private class ModifyParametersWrapper extends HttpServletRequestWrapper {
25         // 获取HTTP参数集合
26         private Map<String, String[]> parameterMap;
27
28         public ModifyParametersWrapper(HttpServletRequest request) {
29             super(request);
30             parameterMap = request.getParameterMap();
31         }
32
33         /**
34          * 重写getParameter方法
35          * @param name
36          * @return
37          */
38         @Override
39         public String getParameter(String name) {
40             String[] results = parameterMap.get(name);
41             if (results == null || results.length <= 0)
42                 return null;
43             else {
44                 String param = results[0];
45                 return "Modify : " + param;
46             }
47         }
48     }
49 }

```

web.xml内容如下

```

1 <!DOCTYPE web-app PUBLIC
2 "-//Sun Microsystems, Inc.//DTD Web Application 2.3//EN"
3 "http://java.sun.com/dtd/web-app_2_3.dtd" >
4
5 <web-app>
6     <display-name>Archetype Created Web Application</display-name>
7     <!-- filter要在servlet前面-->

```

```

8      <filter>
9          <filter-name>ParamFilter</filter-name>
10         <filter-class>org.example.ParamFilter</filter-class>
11     </filter>
12     <filter-mapping>
13         <filter-name>ParamFilter</filter-name>
14         <url-pattern>/*</url-pattern>
15     </filter-mapping>
16
17     <servlet>
18         <servlet-name>login</servlet-name>
19         <servlet-class>org.example.Login</servlet-class>
20     </servlet>
21     <servlet-mapping>
22         <servlet-name>login</servlet-name>
23         <url-pattern>/login</url-pattern>
24     </servlet-mapping>
25 </web-app>

```

通过以上修改后，在Servlet中使用 `request.getParameter("username");` 便会调用 `ModifyParametersWrapper` 对象的 `getParameter` 方法，这就是装饰模式。

此时 `doPost` 方法传的就是 `ModifyParametersWrapper` 对象

`HttpServletRequest` 是 HTTP 请求对象的封装类，`HttpServletResponse` 是 HTTP 响应对象的封装类，各自的主要方法如下图

## 2.3 Servlet四大域对象

| 对象名称             | 作用域                  | 有效时间    |
|------------------|----------------------|---------|
| PageContext对象    | JSP页面（page）          | jsp内有效  |
| ServletRequest对象 | 整个请求（request）        | 一次请求    |
| Session对象        | 一次会话（session）        | 一次会话    |
| ServletContext   | 整个web应用（application） | 整个Web应用 |

域对象共用的方法

```

1  // 存储数据的方法
2  setAttribute(name,value);
3  // 根据name获取对应数据值
4  getAttribute(name);
5  // 删除数据
6  removeAttribute(name);

```

**PageContext对象** 其实是属于JSP的内置对象，它getAttribute()只能获取自己域中保存的属性，而findAttribute()则会按照pageContext->request->session->servletContext的顺序查找有无对应的属性，pageContext对象可以获取8大隐式对象：

- getException()返回Exception。
- getPage()返回Page。
- getRequest()返回request。
- getResponse()返回response。
- getServletConfig()返回config。
- getServletContext()返回application。
- getSession()返回session。
- getOut()返回out。

**ServletContext对象** 在Web应用创建时被创建，当Web应用被停止时停止，所有的Servlet共享一个ServletContext对象，因此可以通过ServletContext实现Servlet之间的通信

**Session对象** 是服务器给每个用户的浏览器创建的独有的Session对象，用于存储用户私有的数据

**ServletRequest对象** 在Servlet的Service方法传入，当请求结束时，被销毁

## 2.4 Filter过滤器技术

Filter在前面讲HTTP对象时已经使用过，简单的说明了应用场景。Filter被称为过滤器，它的作用也如同它的名字一样，就是要过滤HTTP请求，也可以说Web资源，可以分为这么几类

- JSP页面
- Servlet
- 静态文件（HTML、CSS、图片）
- 参数的安全过滤
- 权限控制
- .....

加入Filter之后，一个HTTP请求的过程是这样的

### Filter是如何实现拦截的？

可以看到Filter也是一个链式调用的结构，最开始的filterConfig.getFilter()方法获取的是web.xml中定义的Filter，然后进行链式调用。

### Filter如何定义？

1. 步骤一：定义Java类，实现Filter接口，并实现它的 `init`、`doFilter`、`destroy` 方法
2. 步骤二：在web.xml中使用 `<filter>` 和 `<filter-mapping>` 元素对编写的filter类进行注册，并设置它所能拦截的资源

```

1 <filter>
2     <filter-name>ParamFilter</filter-name>
3     <filter-class>org.example.ParamFilter</filter-class>
4 </filter>
5 <filter-mapping>
6     <filter-name>ParamFilter</filter-name>
7     <url-pattern>/*</url-pattern>
8     <servlet-name>login</servlet-name>
9     <dispatcher>REQUEST</dispatcher>
10 </filter-mapping>

```

**<filter>** 标签主要有以下几个子标签

- 过滤器名称，非空
- 过滤器类的完整路径，非空
- 过滤器初始化参数
  - 指定参数的名称
  - 指定参数的值
  - 在过滤器中，可以使用FilterConfig接口对象来访问初始化参数。

**<filter-mapping>** 标签主要有以下几个子标签

- 设置Filter的名称。该值必须是在元素中声明过的过滤器的名称
- 设置filter所拦截的请求路径（过滤器关联的URL样式）。
- 指定过滤器所拦截的Servlet名称。
- 指定过滤器所拦截的资源被Servlet容器调用的方式，可以是REQUEST，INCLUDE，FORWARD，和ERROR之一，默认是REQUEST。用户可以设置多个子元素用来指定Filter对资源的多种调用方式进行拦截。

**<url-pattern>** 有三种匹配方式和Servlet的配置方式类似

- **绝对路径匹配**：以/开头 不包含通配符 \* 是一个绝对访问路径。例如：/demo、/index.jsp
- **目录匹配**：以/ 开头，以 \* 结尾。例如： /\*、/servlet/\*、/servlet/xxx/\*
- **扩展名匹配**：不能以/开头，也不能以\*结尾，只能以后缀名结尾 例如： \*.do、\*.demo 等，以下都是错误写法

```

1 <url-pattern>/kata/*.jsp</url-pattern>
2 <url-pattern>/*.jsp</url-pattern>
3 <url-pattern>he*.jsp</url-pattern>

```

## 三、Servlet的几点疑问

### 3.1 Servlet是单例模式吗？

Servlet是单例模式，服务器一般情况下只会创建一个Servlet对象，所以ServletContext是作用于整个Web程序

Servlet每收到一个HTTP请求便会产生一个新的线程去处理，但每次的HttpServletRequest对象都是新的

### 3.2 Servlet是线程安全吗？

什么情况下会产生线程安全问题？

1. 存在多线程同时访问一个资源
2. 访问非方法内部的局部变量

线程安全问题都是由全局变量及静态变量引起的，所以使用doGet、doPost方法内部的变量不会出现线程安全问题

修改下Servlet的项目，替换为下面代码，模拟2个HTTP请求，修改mark属性

```
1 public class Login extends HttpServlet {
2     // 模拟访问资源
3     private int mark = 0;
4     @Override
5     protected void doPost(HttpServletRequest request, HttpServletResponse response)
6     throws ServletException, IOException {
7         String userName = request.getParameter("username");
8         String password = request.getParameter("password");
9
10        for(int i = 0;i < 10;i++){
11            try {
12                mark+=1;
13                TimeUnit.SECONDS.sleep(1);
14                System.out.println("当前线程" + userName + "： " + mark);
15            } catch (InterruptedException e) {
16                throw new RuntimeException(e);
17            }
18        }
19        System.out.println("此时的Mark为： " + mark);
20        if (userName.equals("admin") && password.equals("admin")){
21            System.out.println("Login success.");
22        }
23    }
```

浏览器打开两个窗口分别访问首页，分别输入用户名为1和用户名为2，点击提交，结果如下（每次结果略有不同），可以发现mark属性被两次请求互相修改，产生多线程安全问题

```
1 当前线程1： 1
2 当前线程1： 3
```

```
3  当前线程2: 4
4  当前线程1: 5
5  当前线程2: 6
6  当前线程1: 7
7  当前线程2: 8
8  当前线程1: 9
9  当前线程2: 10
10 当前线程1: 11
11 当前线程2: 12
12 当前线程1: 13
13 当前线程2: 14
14 当前线程1: 15
15 当前线程2: 16
16 当前线程1: 17
17 当前线程2: 18
18 当前线程1: 19
19 此时的Mark为: 19
20 当前线程2: 19
21 当前线程2: 20
22 此时的Mark为: 20
```

### 3.1 Servlet中如何处理多线程安全问题?

有三种方法解决线程安全问题

1. 实现 SingleThreadModel 接口
2. 使用关键字synchronized (this){XXXX}
3. 避免使用实例变量

如果一个Servlet实现了SingleThreadModel接口，Servlet引擎将为每个新的请求 **创建一个单独的Servlet实例**，这将 **引起大量的系统开销**

如果使用synchronized关键字，该代码块同一时刻只能有一个线程执行，其它线程处于等待状态，这将严重影响系统性能和客户体验。而且这个方法也无法解决前面线程安全中的代码案例的问题

所以最好的方法是尽量不使用实例变量，方法内部的变量是每个线程的私有变量，线程之间互不影响，所以它们是线程安全的